MCP resources and project code: mcp

For the most up-to-date reference documentation and specifications of the MCP, see the official MCP site at https://modelcontextprotocol.io/docs/getting-started/intro.

The code for this example project can be found in the ch12/mcp folder. You can use that example by running this command in the root folder of the project repository:

```
$ npm run dev -w ch12/mcp
```

This launches a local MCP server that you can explore using the MCP Inspector tool. The Inspector provides a graphical interface for inspecting available integrations (tools, prompts, and resources) exposed by the server, which is helpful for debugging and experimentation in web applications.

You'll need the Gemini API key or OpenAI key for this to work, as the application uses services from different vendors. These are configured as environment variables:

```
// .env
GOOGLE_API_KEY=<GEMINI_API_SECRET_KEY>
OPENAI_API_KEY=<OPEN_AI_SECRET_KEY>
```

Instructions on how to set up and create secret keys for Google AI and OpenAI are provided in the appendix.

12.4.1 MCP server structure

Let's break down the structure of the example MCP server step by step.

MCP SERVER INITIALIZATION

First, we need a running MCP server object before registering any tools, prompts, or resources.

Listing 12.3 Initializing an MCP server

```
import { McpServer } from '@modelcontextprotocol/sdk/server/mcp.js';

const server = new McpServer({
  name: 'chuck-norris-mcp',
  version: '1.0.0',
});
```

Creates a new MCP server instance named chuck-norris-mcp

This server instance is the entry point where all tools, prompts, and resources are registered and later exposed to clients or AI SDKs.

DEFINING A TOOL

Tools provide external functionality that the AI model can invoke dynamically, such as fetching data from APIs. We use the `tool` method to provide an implementation of the API call.